

День 1. Карта мира IT и язык вакансий

Разбираемся, кто работает в IT и как читать первые строки вакансии.

Главная идея

IT — это не одна профессия, а целая отрасль: программы создают разработчики, качество проверяют тестировщики, требования уточняют аналитики, а стабильную работу сервисов поддерживают DevOps-инженеры. Все они решают разные части одной задачи.

Когда пригодится

Вакансия обычно описывает продукт, роль, задачи, обязательные навыки и желательные навыки. Не нужно соответствовать каждому пункту: для junior-позиции важнее крепкая база, способность учиться и несколько законченных учебных проектов.

10 терминов дня

IT	отрасль, связанная с созданием, хранением и обработкой информации
developer	специалист, который проектирует и пишет программный код
QA	специалист по качеству, который ищет ошибки и проверяет требования
analyst	специалист, который переводит потребности бизнеса в понятные требования
DevOps	подход и роль, связывающие разработку, инфраструктуру и доставку продукта
product	цифровой сервис или программа, которая решает задачу пользователя
frontend developer	разработчик пользовательской части продукта
backend developer	разработчик серверной логики и данных
project manager	специалист, который организует работу команды и сроки
junior	начальный профессиональный уровень специалиста

Мини-пример + чек-лист

Схема/команды дня:

IT — это не одна профессия, а целая отрасль: программы создают р...
Вакансия обычно описывает продукт, роль, задачи, обязательные на...
Слово stack, или «стек», означает набор технологий проекта. Напр...

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Откройте любую junior-вакансию на разработчика и выпишите отдельно: задачи, обязательные навыки и стек. Пока ничего не учите наизусть — просто научитесь видеть структуру.

Мини-тест

Что означает слово «стек» в IT-вакансии?

Ответ: Набор технологий, используемых в проекте

Запомни: Стек — это совокупность технологий проекта: язык, фреймворк, база данных и другие инструменты.

День 2. Как устроен компьютер

Понимаем, где выполняется программа и чем память отличается от диска.

Главная идея

Процессор выполняет инструкции программы. Оперативная память временно хранит данные работающих приложений, а накопитель сохраняет файлы надолго. Если выключить компьютер, содержимое RAM исчезнет, а данные на SSD останутся.

Когда пригодится

Операционная система управляет устройствами, файлами, памятью и запущенными программами. Windows, Linux и macOS дают приложениям общий способ обращаться к ресурсам компьютера.

10 терминов дня

CPU	процессор, который выполняет инструкции программ
RAM	быстрая временная память для работающих программ
storage	накопитель для долговременного хранения файлов
operating system	система, управляющая оборудованием и приложениями
process	экземпляр программы, который сейчас выполняется
file system	правила организации файлов и папок на накопителе
core	отдельное вычислительное ядро процессора
cache	очень быстрая память рядом с процессором
SSD	быстрый накопитель без движущихся деталей
driver	программа для взаимодействия системы с устройством

Мини-пример + чек-лист

Схема/команды дня:
Процессор выполняет инструкции программы. Оперативная память вре...
Операционная система управляет устройствами, файлами, памятью и ...
Запущенная программа называется процессом. Один и тот же файл пр...

☐ объяснить тему своими словами

☐ переписать пример кода руками

☐ выполнить практику

☐ ответить на мини-тест

☐ найти одну возможную ошибку

Практика

Откройте диспетчер задач или мониторинг системы. Найдите процесс браузера, посмотрите использование CPU и RAM и объясните своими словами, почему открытые вкладки занимают память.

Мини-тест

Что произойдёт с обычными данными в RAM после выключения компьютера?
Ответ: Они будут потеряны

Запомни: RAM — энергозависимая временная память; после выключения её обычное содержимое теряется.

День 3. Интернет, клиент и сервер

Прослеживаем путь запроса от телефона до нужного сервера.

Главная идея

Интернет — сеть сетей. Ваш телефон или браузер выступает клиентом: он отправляет запрос. Сервер принимает запрос, выполняет работу и возвращает ответ.

Когда пригодится

IP-адрес помогает доставить данные нужному устройству, а DNS переводит понятное имя сайта в IP-адрес. Порт указывает, какой именно программе на сервере предназначено соединение.

10 терминов дня

client	программа или устройство, которое запрашивает услугу
server	программа или компьютер, который обрабатывает запросы
IP address	сетевой адрес устройства для доставки данных
DNS	система, которая преобразует доменное имя в IP-адрес
port	номер конкретной сетевой службы на устройстве
protocol	согласованный набор правил обмена данными
domain	читаемое человеком имя интернет-ресурса
router	устройство, направляющее пакеты между сетями
packet	небольшая порция данных, передаваемая по сети
latency	задержка между отправкой запроса и получением ответа

Мини-пример + чек-лист

Схема/команды дня:

Интернет — сеть сетей. Ваш телефон или браузер выступает клиентом...
IP-адрес помогает доставить данные нужному устройству, а DNS пер...
Протокол — набор правил общения. Он определяет формат сообщений ...

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Выполните команду `nslookup example.com` или откройте любой онлайн DNS lookup. Сравните доменное имя и полученный IP-адрес.

Мини-тест

Зачем нужен DNS?

Ответ: Чтобы переводить имя сайта в IP-адрес

Запомни: DNS позволяет использовать понятные доменные имена вместо запоминания числовых IP-адресов.

День 4. Как работает Web: URL, HTTP и HTTPS

Учимся понимать адрес страницы, запрос и ответ браузера.

Главная идея

URL — полный адрес ресурса. В нём можно увидеть протокол, домен, путь и параметры. Браузер использует этот адрес, чтобы сформировать HTTP-запрос к серверу.

Когда пригодится

Запрос содержит метод, адрес, заголовки и иногда тело. Ответ содержит статус, заголовки и данные. Код 200 означает успех, 404 — ресурс не найден, а 500 — ошибка на сервере.

10 терминов дня

URL	полный адрес ресурса в интернете
HTTP	протокол обмена данными между веб-клиентом и сервером
HTTPS	защищённый шифрованием вариант HTTP
request	сообщение клиента с просьбой выполнить действие
response	ответ сервера со статусом и данными
status code	числовой результат обработки HTTP-запроса
HTTP method	действие запроса, например GET или POST
header	служебная информация запроса или ответа
body	основные данные, передаваемые в запросе или ответе
TLS	протокол шифрования защищённого сетевого соединения

Мини-пример + чек-лист

Схема/команды дня:

URL — полный адрес ресурса. В нём можно увидеть протокол, домен,...
Запрос содержит метод, адрес, заголовки и иногда тело. Ответ сод...
HTTPS — это HTTP поверх защищённого соединения. Шифрование мешае...

☐ объяснить тему своими словами

☐ переписать пример кода руками

☐ выполнить практику

☐ ответить на мини-тест

☐ найти одну возможную ошибку

Практика

Откройте инструменты разработчика браузера, вкладку Network, обновите страницу и найдите запрос документа. Посмотрите его URL, метод и статус.

Мини-тест

Какой HTTP-статус обычно означает, что ресурс не найден?
Ответ: 404

Запомни: 404 Not Found сообщает, что сервер доступен, но запрошенный ресурс не найден.

День 5. Как создают программы в команде

Знакомимся с кодом, задачами, версиями, ошибками и релизами.

Главная идея
Исходный код — текст, написанный программистом. Он хранится в репозитории вместе с историей изменений. Большая работа разбивается на небольшие задачи, которые можно проверить отдельно.

Когда пригодится
Разработчик создаёт отдельную ветку, меняет код, запускает проверки и отправляет изменения на review. После одобрения изменения объединяют с основной веткой.

10 терминов дня

source code	текст программы, написанный разработчиком
repository	хранилище проекта и истории его изменений
branch	изолированная линия изменений в репозитории
commit	зафиксированный снимок осмысленного изменения
bug	ошибка, из-за которой программа работает не так, как ожидается
release	подготовленная версия продукта, выпущенная пользователям
version control	система хранения истории изменений проекта
pull request	предложение проверить и объединить изменения
issue	описанная задача или проблема проекта
changelog	список изменений между версиями продукта

Мини-пример + чек-лист

Схема/команды дня:
Исходный код — текст, написанный программистом. Он хранится в ре...
Разработчик создаёт отдельную ветку, меняет код, запускает прове...
Bug — поведение, которое не соответствует ожиданиям. Исправления...

☐ объяснить тему своими словами

☐ переписать пример кода руками

☐ выполнить практику

☐ ответить на мини-тест

☐ найти одну возможную ошибку

Практика
Возьмите знакомое приложение и придумайте одну feature-задачу и один bug. Для каждого напишите ожидаемый результат одним предложением.

Мини-тест
Зачем разработчик создаёт отдельную ветку?
Ответ: Чтобы изолировать изменения до проверки и объединения

Запомни: Ветка позволяет работать над задачей независимо и безопасно объединить результат после проверки.

День 6. Алгоритмы и структуры данных

Понимаем, что программа делает с данными и почему важен порядок действий.

Главная идея

Алгоритм — точная последовательность шагов от входных данных к результату. Хороший алгоритм должен быть понятным, завершаться и выдавать правильный ответ.

Когда пригодится

Структуры данных задают способ хранения информации. Список удобен для последовательности, очередь — для обработки по порядку поступления, а хеш-таблица — для быстрого поиска по ключу.

10 терминов дня

algorithm	конечная последовательность шагов для решения задачи
complexity	оценка роста затрат времени или памяти
data structure	способ организации и хранения данных
stack	структура, где последним забирают последний добавленный элемент
queue	структура, где первым забирают первый добавленный элемент
hash table	структура для быстрого доступа к значению по ключу
array	последовательная структура элементов
tree	иерархическая структура данных с узлами
graph	структура из вершин и связей между ними
Big O	обозначение роста затрат алгоритма

Мини-пример + чек-лист

Схема/команды дня:

Алгоритм — точная последовательность шагов от входных данных к р...
Структуры данных задают способ хранения информации. Список удобе...
Сложность описывает, как растут время и память при увеличении об...

☐ объяснить тему своими словами

☐ переписать пример кода руками

☐ выполнить практику

☐ ответить на мини-тест

☐ найти одну возможную ошибку

Практика

Запишите алгоритм приготовления чая как строгие шаги. Затем добавьте обработку ситуации, когда в чайнике нет воды.

Мини-тест

Какая структура обслуживает элементы в порядке их поступления?
Ответ: Очередь

Запомни: Очередь работает по принципу FIFO: первым пришёл — первым обслужен.

День 7. Python и рабочее окружение

Устанавливаем базовые инструменты и запускаем первую программу.

Главная идея
Python — язык программирования с понятным синтаксисом. Интерпретатор читает код и выполняет его. Файл Python обычно имеет расширение .py.

Когда пригодится
IDE или редактор помогает писать код, но сам код запускает интерпретатор. Терминал позволяет выполнять команды напрямую и видеть результат или ошибку.

10 терминов дня

Python	язык программирования, ориентированный на читаемый код
interpreter	программа, которая выполняет код Python
IDE	редактор с инструментами запуска, подсказок и отладки
terminal	текстовый интерфейс для выполнения команд
venv	изолированное окружение Python для конкретного проекта
pip	инструмент установки Python-пакетов
script	файл с командами программы
REPL	интерактивная среда выполнения команд
library	готовый переиспользуемый программный код
package manager	инструмент установки и обновления зависимостей

Мини-пример + чек-лист

```
print("Привет, IT!")
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика
Создайте файл hello.py, запустите его командой python hello.py и измените текст сообщения. Если Python ещё не установлен, изучите, где редактор предлагает выбрать интерпретатор.

Мини-тест
Зачем проекту виртуальное окружение venv?
Ответ: Для изоляции библиотек и их версий

Запомни: venv отделяет зависимости проекта от системного Python и других проектов.

День 8. Переменные и типы данных

Учимся хранить числа, текст и логические значения.

Главная идея

Переменная связывает понятное имя со значением. В Python тип относится к самому значению, поэтому одной переменной можно присвоить разные типы, хотя без необходимости так делать не стоит.

Когда пригодится

int хранит целые числа, float — дробные, str — текст, bool — True или False. None обозначает отсутствие значения, а не ноль и не пустую строку.

10 терминов дня

variable	имя, связанное со значением в программе
int	тип целых чисел
float	тип чисел с дробной частью
str	тип текстовых строк
bool	логический тип со значениями True и False
None	специальное значение отсутствия данных
type	категория данных и допустимых операций
assignment	сохранение значения в переменной
constant	значение, которое договорились не изменять
casting	явное преобразование значения к другому типу

Мини-пример + чек-лист

```
name = "Анна"
age = 25
is_beginner = True

print(name, age, is_beginner)
```

☐ объяснить тему своими словами

☐ переписать пример кода руками

☐ выполнить практику

☐ ответить на мини-тест

☐ найти одну возможную ошибку

Практика

Создайте переменные name, age, study_minutes и is_beginner. Выведите каждую вместе с её типом.

Мини-тест

Какое значение в Python означает отсутствие данных?
Ответ: None

Запомни: None специально обозначает, что значения нет; это отличается от нуля, пустой строки и False.

День 9. Условия и логика

Научим программу принимать решения.

Главная идея

Выражение сравнения даёт True или False. Операторы ==, !=, >, <, >= и <= сравнивают значения. Один знак = присваивает значение, а два знака == сравнивают.

Когда пригодится

Блок if выполняется, когда условие истинно. elif проверяет следующий вариант, else обрабатывает все оставшиеся случаи. Отступы в Python определяют границы блока.

10 терминов дня

expression	часть кода, которая вычисляется в значение
comparison	проверка отношения между двумя значениями
logical operator	оператор and, or или not для объединения условий
if	ветка, выполняемая при истинном условии
elif	дополнительная проверка после if
else	ветка для всех не подошедших условий
condition	проверка, результат которой истина или ложь
branch	отдельный путь выполнения программы
truthy	значение, воспринимаемое как истина
short-circuit	остановка вычисления логического выражения раньше конца

Мини-пример + чек-лист

```
age = 20
```

```
if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ закрыт")
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Напишите проверку возраста: младше 18, от 18 до 64 и 65 или больше. Проверьте программу на граничных значениях 17, 18, 64 и 65.

Мини-тест

Чем в Python отличается = от ==?

Ответ: = присваивает, а == сравнивает

Запомни: Один знак = используется для присваивания, а == возвращает результат сравнения.

День 10. Циклы

Повторяем действия без копирования одинакового кода.

Главная идея
Цикл `for` перебирает элементы последовательности. `range` создаёт последовательность чисел и часто используется, когда действие нужно повторить заданное количество раз.

Когда пригодится
`while` повторяет блок, пока условие истинно. Важно менять данные условия, иначе получится бесконечный цикл.

10 терминов дня

iteration	одно выполнение тела цикла
for	цикл перебора элементов последовательности
while	цикл, работающий, пока условие истинно
range	объект, задающий последовательность целых чисел
break	оператор немедленного выхода из цикла
continue	оператор перехода к следующей итерации
counter	переменная для подсчёта повторений
infinite loop	цикл, условие завершения которого никогда не наступает
nested loop	цикл, расположенный внутри другого цикла
enumerate	получение индекса и значения при обходе последовательности

Мини-пример + чек-лист

```
total = 0
for number in range(1, 6):
    total += number

print(total)
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика
Выведите числа от 1 до 10, но пропустите 5. Затем посчитайте сумму выведенных чисел.

Мини-тест
Когда цикл `while` может стать бесконечным?
Ответ: Когда условие никогда не становится `False`

Запомни: `while` продолжает работу, пока условие истинно, поэтому оно должно когда-нибудь измениться.

День 11. Списки и кортежи

Храним несколько значений в правильном порядке.

Главная идея

list — изменяемая упорядоченная коллекция. В неё можно добавлять и удалять элементы. Индексы начинаются с нуля, поэтому первый элемент имеет индекс 0.

Когда пригодится

Срез получает часть последовательности: items[1:4] включает индексы 1, 2 и 3, но не 4. Отрицательный индекс -1 обращается к последнему элементу.

10 терминов дня

list	изменяемая упорядоченная коллекция значений
tuple	неизменяемая упорядоченная коллекция
index	номер позиции элемента, начинающийся с нуля
slice	выбранный диапазон элементов последовательности
mutable	способность объекта изменяться после создания
method	функция, принадлежащая объекту, например list.append
element	отдельное значение внутри коллекции
append	добавление элемента в конец списка
length	количество элементов коллекции
comprehension	компактная запись создания новой коллекции

Мини-пример + чек-лист

```
skills = ["Git", "SQL"]
skills.append("Python")
print(skills[0])
print(skills[:2])
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Создайте список из пяти технологий. Добавьте Python, удалите одну технологию и выведите первые три элемента срезом.

Мини-тест

Какой индекс у первого элемента списка Python?
Ответ: 0

Запомни: Индексация последовательностей Python начинается с нуля.

День 12. Словари и множества

Ищем значения по ключу и оставляем только уникальные элементы.

Главная идея

dict хранит пары ключ-значение. Вместо номера позиции используется осмысленный ключ: `user["name"]`. Ключи должны быть хешируемыми, например строками, числами или кортежами.

Когда пригодится

set хранит только уникальные элементы и не обещает привычный порядок. Множество удобно для удаления дублей и быстрых проверок принадлежности.

10 терминов дня

dictionary	коллекция пар ключ-значение
key	уникальный идентификатор значения в словаре
value	данные, сохранённые по ключу
set	неупорядоченная коллекция уникальных элементов
hashable	пригодный для стабильного хеширования неизменяемый объект
membership	проверка наличия элемента оператором in
mapping	соответствие ключей их значениям
unique	свойство значения не повторяться в наборе
get	безопасное получение значения словаря по ключу
intersection	общие элементы двух множеств

Мини-пример + чек-лист

```
user = {"name": "Анна", "level": "junior"}
print(user["name"])

unique_skills = set(["Python", "Git", "Python"])
print(unique_skills)
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Создайте словарь профиля с именем, городом и списком навыков. Затем превратите список навыков с повторами во множество.

Мини-тест

Какая коллекция автоматически убирает повторяющиеся элементы?

Ответ: set

Запомни: Множество set хранит только уникальные элементы.

День 13. Строки и ввод пользователя

Обрабатываем текст и собираем понятные сообщения.

Главная идея

Строка — последовательность символов. Её можно объединять, получать по индексу и обрабатывать методами, но сам объект `str` неизменяем.

Когда пригодится

`f`-строка подставляет выражения внутрь текста и обычно читается лучше склеивания через `+`. `input` получает строку от пользователя, даже если введены цифры.

10 терминов дня

string	последовательность текстовых символов
concatenation	соединение строк в одну
f-string	строка с подстановкой выражений в фигурных скобках
input	функция чтения строки, введённой пользователем
strip	метод удаления пробелов и переносов по краям
split	метод разделения строки на список частей
substring	непрерывная часть строки
delimiter	символ или строка, разделяющие части текста
formatting	подстановка и оформление значений в строке
Unicode	стандарт представления символов разных языков

Мини-пример + чек-лист

```
name = input("Ваше имя: ").strip()
minutes = int(input("Минуты учёбы: "))
print(f"{name}, сегодня вы учились {minutes} минут.")
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Спросите имя и год рождения, вычислите примерный возраст и выведите приветствие одной `f`-строкой.

Мини-тест

Какой тип всегда возвращает функция `input`?
Ответ: `str`

Запомни: `input` возвращает строку; при необходимости её преобразуют в число отдельно.

День 14. Функции

Разбиваем программу на маленькие переиспользуемые части.

Главная идея

Функция объединяет действия под именем. Это помогает не копировать код и позволяет проверять небольшую часть программы отдельно.

Когда пригодится

Параметры указаны в определении функции, аргументы передаются при вызове. `return` возвращает результат вызывающему коду и завершает выполнение функции.

10 терминов дня

function	именованный переиспользуемый блок кода
parameter	имя входного значения в определении функции
argument	конкретное значение, переданное при вызове
return	оператор возврата результата из функции
scope	область программы, где доступно имя
docstring	строка документации внутри функции, класса или модуля
call	запуск функции с указанными аргументами
default argument	значение параметра, используемое без явного аргумента
keyword argument	аргумент, переданный с именем параметра
pure function	функция без побочных эффектов при одинаковом вводе

Мини-пример + чек-лист

```
def calculate_total(price, quantity):
    return price * quantity

result = calculate_total(150, 3)
print(result)
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Напишите функцию `calculate_total(price, quantity)`, которая возвращает стоимость. Вызовите её для трёх товаров.

Мини-тест

В чём разница между параметром и аргументом?
Ответ: Параметр задаётся в функции, аргумент передаётся при вызове

Запомни: Параметр — место для входного значения в определении, аргумент — фактическое значение при вызове.

День 15. Ошибки и отладка

Учимся читать traceback и находить первопричину проблемы.

Главная идея

Синтаксическая ошибка означает, что Python не смог разобрать код. Исключение возникает уже во время выполнения: например, при делении на ноль или обращении к отсутствующему ключу.

Когда пригодится

Traceback показывает цепочку вызовов и строку, где проявилась ошибка. Читать его полезно снизу вверх: сначала тип и сообщение, затем место в своём коде.

10 терминов дня

syntax error	ошибка в структуре кода, мешающая его разобрать
exception	объект ошибки, возникшей во время выполнения
traceback	цепочка вызовов, приведшая к исключению
debugger	инструмент пошагового выполнения и просмотра состояния
try/except	конструкция обработки ожидаемых исключений
raise	оператор явного создания исключения
validation	проверка входных данных до основной обработки
error handling	предусмотренная реакция программы на сбой
logging	запись событий программы для анализа
finally	блок, выполняемый независимо от наличия исключения

Мини-пример + чек-лист

```
try:
    age = int("не число")
except ValueError:
    print("Возраст должен быть числом")
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Специально вызовите ValueError через `int("текст")`, прочитайте последнюю строку traceback, затем обработайте эту конкретную ошибку.

Мини-тест

С какой части traceback обычно удобнее начинать чтение?

Ответ: С последней строки с типом и сообщением ошибки

Запомни: Последняя строка обычно сразу показывает тип и краткую причину ошибки.

День 16. Модули, пакеты и зависимости

Собираем программу из файлов и подключаем готовые библиотеки.

Главная идея

Модуль — обычный Python-файл, который можно импортировать. Пакет объединяет связанные модули в папке. Это помогает делить большой проект по ответственности.

Когда пригодится

Зависимость — сторонний пакет, необходимый проекту. `pip` устанавливает пакет, а файл `requirements.txt` фиксирует список и часто версии зависимостей.

10 терминов дня

module	Python-файл, код которого можно импортировать
package	набор связанных Python-модулей
import	подключение доступного модуля в программу
dependency	внешняя библиотека, от которой зависит проект
requirements	описание зависимостей проекта и их версий
version	идентификатор конкретного выпуска программы или пакета
namespace	область, связывающая имена с объектами
semantic versioning	схема версий вида <code>major.minor.patch</code>
virtual environment	изолированное окружение зависимостей проекта
standard library	модули, поставляемые вместе с языком

Мини-пример + чек-лист

```
from pathlib import Path

project_file = Path("data") / "users.json"
print(project_file)
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Создайте модуль `helpers.py` с одной функцией, импортируйте её в `main.py`. Посмотрите список пакетов командой `pip list`.

Мини-тест

Для чего нужен `requirements.txt`?
Ответ: Для описания зависимостей проекта

Запомни: `requirements.txt` помогает воспроизвести набор Python-зависимостей проекта.

День 17. Файлы, JSON и CSV

Сохраняем данные и читаем распространённые форматы.

Главная идея

Путь указывает расположение файла. Конструкция `with open(...)` автоматически закрывает файл даже при ошибке. Текст нужно читать и записывать в согласованной кодировке, обычно UTF-8.

Когда пригодится

JSON хранит словари, списки, строки, числа и логические значения в текстовом виде. Он часто используется в API и конфигурации.

10 терминов дня

file	именованная область долговременного хранения данных
path	адрес файла или папки в файловой системе
encoding	правило преобразования символов в байты и обратно
JSON	текстовый формат структурированных данных
CSV	текстовый табличный формат с разделителями
serialization	преобразование объекта в формат хранения или передачи
read mode	режим открытия файла для чтения
write mode	режим открытия файла для записи
parser	программа, разбирающая текстовый формат
delimiter	разделитель полей в табличном текстовом файле

Мини-пример + чек-лист

```
import json
```

```
profile = {"name": "Анна", "skills": ["Python", "Git"]}
with open("profile.json", "w", encoding="utf-8") as file:
    json.dump(profile, file, ensure_ascii=False)
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Сохраните словарь профиля в JSON-файл, затем прочитайте его обратно и выведите одно поле.

Мини-тест

Почему полезно открывать текстовые файлы с `encoding="utf-8"`?

Ответ: Чтобы явно и одинаково интерпретировать символы

Запомни: Явная UTF-8-кодировка уменьшает риск неправильного отображения символов на разных системах.

День 18.

Объектно-ориентированное программирование

Понимаем классы и объекты без сложной теории.

Главная идея

Класс описывает общую форму и поведение объектов. Объект — конкретный экземпляр класса со своими данными. Например, User — класс, а пользователь Анна — объект.

Когда пригодится

Атрибут хранит состояние объекта, метод описывает действие. Метод `__init__` инициализирует новый объект и получает его через параметр `self`.

10 терминов дня

class	описание структуры и поведения объектов
object	конкретный экземпляр класса
attribute	данные, принадлежащие объекту
method	функция, определённая внутри класса
constructor	логика начальной настройки нового объекта
inheritance	создание класса на основе поведения другого класса
instance	конкретный объект, созданный из класса
encapsulation	объединение данных и поведения внутри объекта
polymorphism	единый интерфейс для объектов разных типов
super	обращение к реализации родительского класса

Мини-пример + чек-лист

```
class Course:
    def __init__(self, title, days):
        self.title = title
        self.days = days

    def describe(self):
        return f'{self.title}: {self.days} дней'

course = Course("Фундамент IT", 30)
print(course.describe())
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Создайте класс `Course` с названием и количеством дней. Добавьте метод `describe`, возвращающий понятную строку.

Мини-тест

Что является конкретным экземпляром класса?
Ответ: Объект

Запомни: Объект — конкретный экземпляр, созданный по описанию класса.

День 19. Git и совместная работа

Сохраняем историю кода и безопасно объединяем изменения.

Главная идея

Git — распределённая система контроля версий. Репозиторий хранит историю локально, поэтому коммиты можно создавать без постоянного подключения к серверу.

Когда пригодится

Коммит должен быть небольшим осмысленным изменением с понятным сообщением. Ветка позволяет выполнять задачу отдельно, merge объединяет истории.

10 терминов дня

Git	система контроля версий исходного кода
repository	проект с историей изменений Git
commit	сохранённое осмысленное состояние изменений
branch	отдельная линия разработки
merge	объединение изменений из разных веток
remote	связанный удалённый репозиторий
working tree	текущие файлы проекта на компьютере
staging area	подготовленный набор изменений для коммита
clone	создание локальной копии удалённого репозитория
conflict	несовместимые изменения, требующие ручного выбора

Мини-пример + чек-лист

```
git init
git add README.md
git commit -m "Добавить описание проекта"
git log --oneline
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Создайте тестовый репозиторий, добавьте README, выполните git add и git commit. Посмотрите историю через git log --oneline.

Мини-тест

Что делает commit в Git?

Ответ: Фиксирует осмысленный снимок изменений

Запомни: Коммит сохраняет выбранные изменения и добавляет их в историю репозитория.

День 20. Командная строка и Linux

Уверенно перемещаемся по папкам и понимаем серверную среду.

Главная идея

Shell принимает текстовые команды и запускает программы. pwd показывает текущую папку, ls — содержимое, cd — переход, mkdir — создание каталога.

Когда пригодится

Абсолютный путь начинается от корня, относительный — от текущей папки. Перед изменением или удалением файлов всегда полезно проверить, где вы находитесь.

10 терминов дня

shell	программа, принимающая и выполняющая текстовые команды
command	указание операционной системе выполнить действие
directory	папка файловой системы
absolute path	полный путь от корня файловой системы
environment variable	внешняя именованная настройка процесса
permission	правило доступа к файлу или действию
home directory	личная папка пользователя в системе
relative path	путь, отсчитываемый от текущей директории
pipe	передача вывода одной команды на вход другой
sudo	выполнение команды с повышенными правами

Мини-пример + чек-лист

```
pwd
ls
mkdir it-practice
cd it-practice
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Создайте папку it-practice из терминала, перейдите в неё, создайте вложенную папку day-20 и выведите текущий абсолютный путь.

Мини-тест

Почему секреты часто передают через переменные окружения?
Ответ: Чтобы не записывать их прямо в исходный код

Запомни: Переменные окружения отделяют настройки и секреты от исходного кода и его истории.

День 21. Базы данных и SQL

Храним связанные данные надёжнее, чем в случайных файлах.

Главная идея

База данных организует хранение и поиск информации. В реляционной базе данные находятся в таблицах, состоящих из строк и колонок.

Когда пригодится

Строка обычно представляет один объект, колонка — одно его свойство. Первичный ключ уникально определяет строку и позволяет связывать данные.

10 терминов дня

database	организованное хранилище данных с правилами доступа
table	структура строк и колонок одного типа сущностей
row	одна запись таблицы
column	именованное свойство записей таблицы
primary key	уникальный идентификатор строки
SQL	язык работы с реляционными базами данных
schema	структура и правила организации базы данных
foreign key	ссылка одной таблицы на запись другой
constraint	ограничение допустимых данных таблицы
NULL	отсутствующее или неизвестное значение в базе

Мини-пример + чек-лист

CREATE TABLE courses (
id INTEGER PRIMARY KEY,
title TEXT NOT NULL,
days INTEGER NOT NULL
);

☐ объяснить тему своими словами

☐ переписать пример кода руками

☐ выполнить практику

☐ ответить на мини-тест

☐ найти одну возможную ошибку

Практика

Нарисуйте таблицу courses с колонками id, title, days и is_active. Отметьте тип каждой колонки и первичный ключ.

Мини-тест

Для чего нужен primary key?
Ответ: Для уникальной идентификации строки

Запомни: Первичный ключ однозначно определяет запись и используется в связях.

День 22. SQL-запросы, связи и индексы

Выбираем нужные данные и понимаем цену быстрого поиска.

Главная идея
SELECT выбирает данные, WHERE фильтрует строки, ORDER BY сортирует. Запрос должен запрашивать только необходимые колонки и строки.

Когда пригодится
JOIN соединяет строки связанных таблиц. Например, user_id в таблице результатов указывает на id пользователя. Тип JOIN определяет, что делать с отсутствующими связями.

10 терминов дня

query	формальный запрос к базе данных
SELECT	SQL-команда чтения данных
WHERE	условие фильтрации строк
JOIN	операция соединения связанных таблиц
index	дополнительная структура для ускорения поиска
transaction	группа операций с принципом всё или ничего
INSERT	SQL-команда добавления новых строк
UPDATE	SQL-команда изменения существующих строк
DELETE	SQL-команда удаления строк
rollback	отмена изменений незавершённой транзакции

Мини-пример + чек-лист

```
SELECT title, days
FROM courses
WHERE is_active = 1 AND days = 30
ORDER BY title;
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика
Напишите SELECT, который выбирает активные курсы продолжительностью 30 дней и сортирует их по названию.

Мини-тест
Какое свойство транзакции защищает от частично выполненной группы операций?
Ответ: Все операции выполняются или откатываются вместе

Запомни: Транзакция не оставляет согласованную группу операций выполненной только наполовину.

День 23. API, REST и JSON

Связываем программы через понятный контракт.

Главная идея

API — согласованный интерфейс, через который одна программа использует возможности другой. В веб-API клиент обращается к endpoint по HTTP.

Когда пригодится

REST — распространённый стиль: ресурс имеет URL, а действие выражается методом. GET читает, POST создаёт, PUT или PATCH изменяет, DELETE удаляет.

10 терминов дня

API	согласованный интерфейс взаимодействия программ
endpoint	конкретный адрес операции веб-API
REST	стиль проектирования API вокруг ресурсов и HTTP-методов
JSON	текстовый формат структурированных данных
authentication	проверка того, кто выполняет запрос
token	строка, подтверждающая право клиента на запрос
HTTP status	числовой результат обработки API-запроса
payload	полезные данные запроса или ответа
rate limit	ограничение числа запросов за период
idempotency	свойство повторного запроса не менять результат дополнительно

Мини-пример + чек-лист

```
{
  "course": "IT Foundation",
  "day": 23,
  "passed": true
}
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Спроектируйте три endpoint для курсов: список, конкретный курс и сохранение результата. Укажите HTTP-метод каждого.

Мини-тест

Какой HTTP-метод обычно используют для чтения ресурса без изменения?
Ответ: GET

Запомни: GET предназначен для получения представления ресурса без его изменения.

День 24. Frontend, backend и полный путь данных

Понимаем части веб-приложения и их ответственность.

Главная идея
Frontend выполняется у пользователя и отвечает за интерфейс. HTML задаёт структуру, CSS — оформление, JavaScript — интерактивное поведение.

Когда пригодится
Backend работает на сервере: проверяет права, применяет бизнес-правила, обращается к базе и формирует ответ. Python часто используется именно на backend.

10 терминов дня

frontend	часть приложения, отвечающая за пользовательский интерфейс
backend	серверная часть с логикой, доступом и данными
HTML	язык структуры веб-страницы
CSS	язык визуального оформления веб-страницы
JavaScript	язык поведения веб-интерфейса
framework	готовый каркас и набор инструментов разработки
DOM	объектное представление HTML-страницы
responsive design	адаптация интерфейса к разным экранам
component	самостоятельная переиспользуемая часть интерфейса
build tool	инструмент подготовки исходного кода к запуску

Мини-пример + чек-лист

```
print("Практика дня")
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика
Возьмите форму входа и разделите её путь на шаги frontend, HTTP-запрос, backend, база данных и HTTP-ответ.

Мини-тест
Где обычно проверяют пароль и права пользователя?
Ответ: На backend

Запомни: Критические проверки выполняют на backend, которому клиент не может диктовать результат.

День 25. Первое веб-приложение на Flask

Связываем Python-функцию с URL и HTML-шаблоном.

Главная идея

Flask — небольшой Python-фреймворк для веб-приложений. Route связывает URL и HTTP-методы с view-функцией.

Когда пригодится

View получает данные запроса, вызывает нужную логику и возвращает response. Шаблон отделяет HTML от Python и безопасно подставляет подготовленные значения.

10 терминов дня

Flask	лёгкий Python-фреймворк для веб-приложений
route	правило связи URL с обработчиком
view	функция, обрабатывающая веб-запрос
template	HTML-заготовка с безопасными местами для данных
request	объект входных данных текущего HTTP-запроса
session	состояние пользователя между несколькими запросами
blueprint	способ группировки маршрутов Flask
context	данные, передаваемые шаблону для отрисовки
cookie	небольшие данные, сохраняемые браузером
middleware	слой обработки между запросом и приложением

Мини-пример + чек-лист

```
from flask import Flask

app = Flask(__name__)

@app.get("/hello/<name>")
def hello(name):
    return f"Привет, {name}!"
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Нарисуйте маршрут /hello/<name>: какие данные входят, что делает view и какой HTML должен вернуться.

Мини-тест

Какова задача view-функции во Flask?

Ответ: Обработать запрос и сформировать ответ

Запомни: View связывает HTTP-запрос с логикой приложения и возвращает ответ.

День 26. Тестирование

Доказываем поведение программы и защищаем его от повторных ошибок.

Главная идея
Тест автоматически выполняет код с заданными входами и сравнивает результат с ожиданием.
Хороший тест проверяет одно понятное поведение.

Когда пригодится
Unit-тест изолирует небольшую функцию, integration-тест проверяет взаимодействие нескольких частей. Пирамида тестов обычно содержит много быстрых unit-тестов и меньше крупных сценариев.

10 терминов дня

test case	один автоматический сценарий проверки поведения
unit test	изолированная проверка небольшой единицы кода
integration test	проверка совместной работы нескольких компонентов
assert	утверждение об ожидаемом результате
mock	контролируемая замена внешней зависимости в тесте
coverage	метрика кода, выполненного во время тестов
fixture	подготовленные данные и окружение для теста
regression test	тест против повторного появления исправленной ошибки
test suite	набор связанных автоматических тестов
edge case	редкий пограничный сценарий работы программы

Мини-пример + чек-лист

```
def calculate_total(price, quantity):  
    return price * quantity  
  
def test_calculate_total():  
    assert calculate_total(150, 3) == 450
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика
Напишите три теста для calculate_total: обычные значения, нулевое количество и отрицательное значение, если оно запрещено правилами.

Мини-тест
Гарантирует ли 100% coverage отсутствие ошибок? Ответ: Нет, coverage не оценивает качество ожиданий

Запомни: Покрытие показывает выполнение строк, но тест может не проверять важный результат или пограничный случай.

День 27. Чистый код и code review

Пишем код, который сможет понять другой человек и будущий вы.

Главная идея

Читаемость важнее хитрости. Имена должны объяснять смысл, функции — делать одну понятную вещь, а комментарии — отвечать «почему», если это нельзя выразить кодом.

Когда пригодится

DRY напоминает не дублировать знание, но раннее объединение непохожих случаев тоже вредит. Refactoring улучшает структуру без изменения внешнего поведения и должен поддерживаться тестами.

10 терминов дня

naming	выбор имён, раскрывающих назначение данных и действий
DRY	принцип устранения повторяющегося знания
refactoring	улучшение структуры без изменения поведения
code review	проверка изменений другим разработчиком
linter	автоматический анализатор стиля и типовых ошибок
technical debt	будущая стоимость принятых сегодня упрощений
readability	насколько легко человеку понять код
duplication	повторение одинаковой логики в нескольких местах
code smell	признак потенциальной проблемы в устройстве кода
formatter	инструмент автоматического единообразного оформления кода

Мини-пример + чек-лист

```
print("Практика дня")
```

☐ объяснить тему своими словами

☐ переписать пример кода руками

☐ выполнить практику

☐ ответить на мини-тест

☐ найти одну возможную ошибку

Практика

Возьмите старую функцию или пример с плохими именами x, a, do_it. Переименуйте сущности и разделите код так, чтобы комментарий о действиях стал не нужен.

Мини-тест

Что должно сохраняться при рефакторинге?
Ответ: Внешнее поведение программы

Запомни: Рефакторинг меняет внутреннюю структуру, сохраняя наблюдаемое поведение.

День 28. Безопасность приложений

Различаем вход, права и безопасное хранение секретов.

Главная идея

Authentication отвечает на вопрос «кто вы?», authorization — «что вам разрешено?». Успешный вход не означает право видеть чужие данные.

Когда пригодится

Пароли хранят не открытым текстом, а в виде медленного криптографического хеша с солью. Секретные ключи не размещают в исходном коде и публичном репозитории.

10 терминов дня

vulnerability	слабое место, которое можно использовать во вред
hash	одностороннее преобразование данных в фиксированное значение
authentication	проверка личности пользователя
authorization	проверка разрешения на конкретное действие
injection	внедрение пользовательского ввода в команду или запрос
secret	закрытое значение вроде ключа, токена или пароля
encryption	обратимое преобразование данных с секретным ключом
CSRF	атака с выполнением запроса от имени вошедшего пользователя
XSS	внедрение вредоносного скрипта в веб-страницу
least privilege	выдача только минимально необходимых прав

Мини-пример + чек-лист

```
cursor.execute(
    "SELECT id, title FROM courses WHERE user_id = ?",
    (current_user_id,)
)
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Проверьте учебный проект: нет ли паролей в коде, параметризованы ли SQL-запросы, проверяется ли владелец данных, ограничены ли сообщения об ошибках.

Мини-тест

В чём разница между authentication и authorization? Ответ: Первая проверяет личность, вторая — право на действие

Запомни: Сначала система устанавливает личность, затем отдельно проверяет разрешение на ресурс или действие.

День 29. Деплой, Docker, облако и CI/CD

Понимаем, как код превращается в работающий сервис.

Главная идея

Deploy — доставка проверенной версии в среду, где ей пользуются. Сервер может быть физическим или виртуальным, а cloud предоставляет вычисления, сети и хранилища как услугу.

Когда пригодится

Контейнер упаковывает приложение с окружением. Docker создаёт и запускает контейнеры по описанию Dockerfile. Контейнер не является полноценной виртуальной машиной и использует ядро хост-системы.

10 терминов дня

deploy	доставка версии приложения в целевую среду
server	система, предоставляющая сервис другим программам
cloud	удалённые вычислительные ресурсы как услуга
container	изолированный пакет приложения и его окружения
Docker	платформа сборки и запуска контейнеров
CI/CD	автоматизация проверок, подготовки и доставки изменений
image	шаблон файлов и настроек для создания контейнера
pipeline	автоматическая последовательность сборки и проверки
environment	окружение с настройками и ресурсами приложения
rollback	возврат развёртывания к рабочей версии

Мини-пример + чек-лист

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Нарисуйте путь commit → тесты CI → образ Docker → сервер → проверка здоровья. Для каждого шага укажите, какая ошибка может остановить выпуск.

Мини-тест

Что обязательно должно происходить до автоматического деплоя?
Ответ: Успешные проверки и сборка

Запомни: Конвейер должен остановить доставку, если код не проходит тесты или сборку.

День 30. Профессия, портфолио и следующий шаг

Собираем знания в понятный план выхода на junior Python-разработчика.

Главная идея

Junior — не человек, который знает всё, а разработчик, способный решать ограниченные задачи с обратной связью. Middle работает самостоятельнее и лучше оценивает последствия решений.

Когда пригодится

Портфолио должно показывать законченные проекты: понятный README, рабочий запуск, Git-историю, тесты и объяснение архитектуры. Один аккуратный сервис полезнее десятка незавершённых папок.

10 терминов дня

stack	набор технологий, используемых в проекте
junior	начальный уровень специалиста, работающего с поддержкой
middle	уровень специалиста с большей самостоятельностью
soft skills	навыки общения, обратной связи и организации работы
portfolio	набор проектов, демонстрирующих практические навыки
interview	встреча для взаимной оценки кандидата и компании
resume	краткое описание опыта и навыков кандидата
roadmap	последовательный план профессионального развития
mentor	опытный специалист, помогающий в обучении
feedback	конкретная обратная связь о результате работы

Мини-пример + чек-лист

```
print("Практика дня")
```

- ☐ объяснить тему своими словами
- ☐ переписать пример кода руками
- ☐ выполнить практику
- ☐ ответить на мини-тест
- ☐ найти одну возможную ошибку

Практика

Составьте план проекта «Трекер обучения»: Flask, SQLite, авторизация, CRUD уроков, API прогресса, тесты и README. Разбейте работу минимум на десять небольших задач.

Мини-тест

Что лучше всего показывает готовность junior-разработчика к практике?
Ответ: Законченный понятный проект с кодом, тестами и README

Запомни: Завершённый проект показывает, что кандидат умеет доводить задачу до работающего и объяснимого результата.